

has worked, list all the currently loaded modules:

```
[root@vbg modules]# semodule -l
ccs 1.0.0
clamav 1.1.0
doc 1.1.0
evolution 1.1.0
iscsid 1.0.0
mozilla 1.1.0
mplayer 1.1.0
nagios 1.1.0
odkjob 1.0.1
openoffice 1.0.0
pocod 1.0.0
pyzor 1.1.0
razor 1.1.0
rnci 1.0.0
smartmon 1.1.0
tvp 1.0.1
vbg 1.0.3
```

To understand the difference made by the removal of the 'amavis' package, again redirect the output of *seinfo* to a file:

```
[root@vbg modules]# seinfo > /tmp/new-selinux-policy
```

...and run a *diff* on the two files:

```
[root@vbg modules]# diff /tmp/org-selinux-policy /tmp/new-selinux-policy
6c6
< Types: 1516 Attributes: 148
---
> Types: 1507 Attributes: 148
8c8
< Booleans: 211 Cond. Expr.: 187
---
> Booleans: 210 Cond. Expr.: 186
10,11c10,11
< Allow: 82576 Neverallow: 0
< Auditallow: 28 Dontaudit: 5066
---
> Allow: 81929 Neverallow: 0
> Auditallow: 28 Dontaudit: 5062
13c13
< Type trans: 1400 Type change: 17
---
> Type trans: 1387 Type change: 17
```

You will see that removing *amavis* has made the following changes:

1. Reducing the number of 'types'

from 1,516 to 1,507

2. Reducing the number of 'Booleans' from 211 to 210
3. Reducing the number of 'allow rules' from 82,576 to 81,929
4. Reducing the number of 'type transition rules' from 1400 to 1,387 and so on...

Thus, we see that by using modules, we can at least add types, Booleans and rules to the core policy. That is pretty much what we want to do when we need to modify the default policy to suit our needs.

From what we've just covered, it is clear that we need to create SELinux policy modules—thereby creating new types, Booleans and various rules.

Creating SELinux modules

SELinux policy modules need to be written in the SELinux policy language. It is not a complicated language at all, but like most programming languages, requires a certain structure and syntax to be followed while creating the modules.

Also, once the text files containing our desired modifications have been created, we need to compile them into a binary policy module (Policy Package). Once the Policy Package files have been created, they just need to be tested and then finally loaded to enable the desired functionality.

To enable the development and compilation of policy modules, install the *selinux-policy-devel* RPM. The installed RPM on my system is *selinux-policy-devel-2.4.6-106.el5_1.3*.

This module creates the */usr/share/selinux/devel/* directory, which contains 'include' files and a makefile for compilation. It also installs three files—*example.te*, *example.fc* and *example.if*—to assist you in the creation of policy modules.

The three files are important to understand the structure of policy modules:

- The type enforcement file (a file with the *.te* extension—for example, */usr/share/selinux/devel/example.te*) is the most important file. This contains the

name of the module, its version and all the additions desired in the policy, such as types, rules, Booleans, etc.

- The file contexts file (with the *.fc* extension—for example, */usr/share/selinux/devel/example.fc*) contains the default security contexts to be provided for files created/used by the application for which we are creating the policy module.
- The interface file (one with the *.if* extension—for example, */usr/share/selinux/devel/example.if*) generally would contain macro definitions that assist in creating type enforcement rules.

The type enforcement file (*.te*) is mandatory, while the other two files (*.fc* and *.if*), if not required, need not be explicitly created. I would advise their use, but it generally depends on the kind of policy module to be developed.

Syntax of the type enforcement file

The most important thing for a policy module to be clearly distinguished is its name and version—the output of the *semodule -l* command. This is specified as the first line in a *.te* file as *policy_name(<policy name>, <policy version>)*.

New types being introduced by the module are declared as *type <new type>;*.

Let's create a small policy module called *test* to introduce a new type called *lfy_t*. Use the following steps to achieve the above:

1. Create a work directory for building and compiling your SELinux modules.
2. Copy the necessary files needed for compiling SELinux modules.
3. Create at least a *.te* file for your SELinux module, specifying the module name and version number.
4. Compile the source file above to a binary policy package file.
5. Load the binary policy package.
6. Test the changes in the SELinux Policy.